

Safe State

Definition:

If the system can successfully allocate resources to each process by giving resources, then it is called a **safe state**.

Given:

Total Resource (R1) = **12**

Process	Required Resource	Allocated Resource	Resource Needed
P1	6	2	4
P2	11	5	6
P3	5	3	2

Total Allocated Resource = **2 + 5 + 3 = 10**

Available Resource =

Total – Allocated = 12 – 10 = 2

Safe Sequence:

P3 → P1 → P2

Unsafe State

Definition:

If the system cannot avoid deadlock by allocating resources to each process, then it is called an **unsafe state**.

Process	Required Resource	Allocated Resource	Resource Needed
P1	9	2	7
P2	11	5	6
P3	5	3	2

Total Allocated Resource = **10**

Available Resource = **12 – 10 = 2**

- Only **P3** can execute first (needs 2)
- After that, remaining processes **cannot proceed safely**
- Leads to **deadlock possibility**

Difference

Safe State	Unsafe State
All processes can finish	Some processes may get stuck
Safe sequence exists	No safe sequence
No deadlock	Deadlock possible

Resource Allocation Graph (RAG)

A **Resource Allocation Graph (RAG)** represents how processes and resources interact in a system. It helps analyze situations like **deadlock**.

1. Vertex Types

In a RAG, there are two types of nodes (vertices):

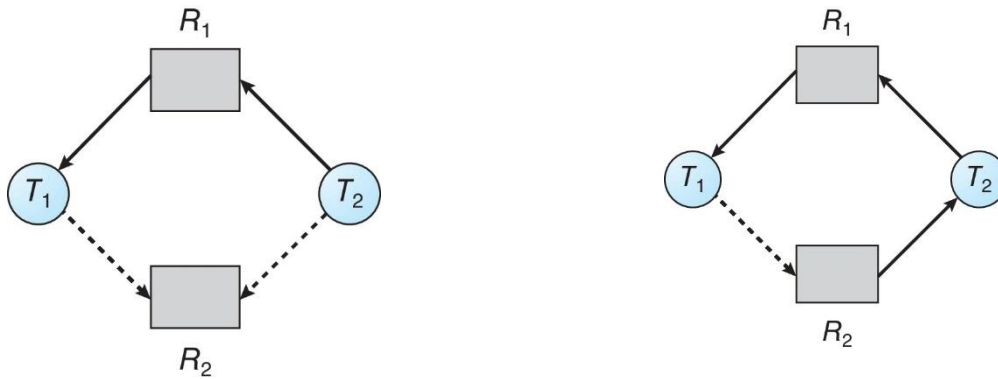
- **Resource (R)**
Represented by a **rectangle**
Examples: CPU, Memory, Disk
- **Process (P)**
Represented by a **circle**
Represents active processes running in the system

2. Edge Types

Edges (arrows) show the relationship between processes and resources:

- **Request Edge ($P \rightarrow R$)**
A process is requesting or waiting for a resource
- **Allocated / Assignment Edge ($R \rightarrow P$)**
A resource is currently allocated to a process
- **Claim Edge ($P \rightsquigarrow R$) (dashed line)**
A process may request this resource in the future

3. Example Scenario



Banker's Algorithm

Banker's Algorithm is a **deadlock avoidance algorithm** used in operating systems. It ensures that resource allocation is done in such a way that the system always remains in a **safe state**.

Key Concepts

1. **Available**
Number of available resources of each type.
2. **Max**
Maximum demand of each process.
3. **Allocation**
Resources currently allocated to each process.
4. **Need**
Remaining resources required by a process.
 $\text{Need} = \text{Max} - \text{Allocation}$

Steps of Banker's Algorithm

Step 1: Initialize

- Find **Available**, **Max**, and **Allocation**
- Calculate **Need** = **Max** – **Allocation**

Step 2: Find a Safe Process

- Pick a process whose **Need** $\leq$ **Available**
- If no such process exists $\rightarrow$ Unsafe state

Step 3: Allocate Resources

- Assume the process finishes
- Release its allocated resources back:
- Available = Available + Allocation

Step 4: Repeat

- Continue until all processes are completed

Step 5: Safe Sequence

- If all processes finish → Safe State
- The order is called **Safe Sequence**

System is in **Safe State**

Safe Sequence:

P1 → P3 → P4 → P2 → P0

Deadlock Avoidance

Here, R1 = 10, R2 = 5, R3 = 7

Process	Max Need (R1 R2 R3)	Allocated Resource (R1 R2 R3)	Current Resource Need (R1 R2 R3)	Available Resource (R1 R2 R3)
P0	7 5 3	0 1 0	7 4 3	3 3 2
P1	3 2 2	2 0 0	1 2 2	5 3 2
P2	9 0 2	3 0 2	6 0 0	7 4 3
P3	2 2 2	2 1 1	0 1 1	7 4 5
P4	4 3 3	0 0 2	4 3 1	10 4 7
		7 2 5		10 5 7

Current Resource Need = Max Need – Allocated Resource

Safe Sequence:

P1 → P3 → P4 → P2 → P0

Deadlock Detection

1. Find Available resources
2. Look for a process where:

$$\text{Request} \leq \text{Available}$$

3. If found:

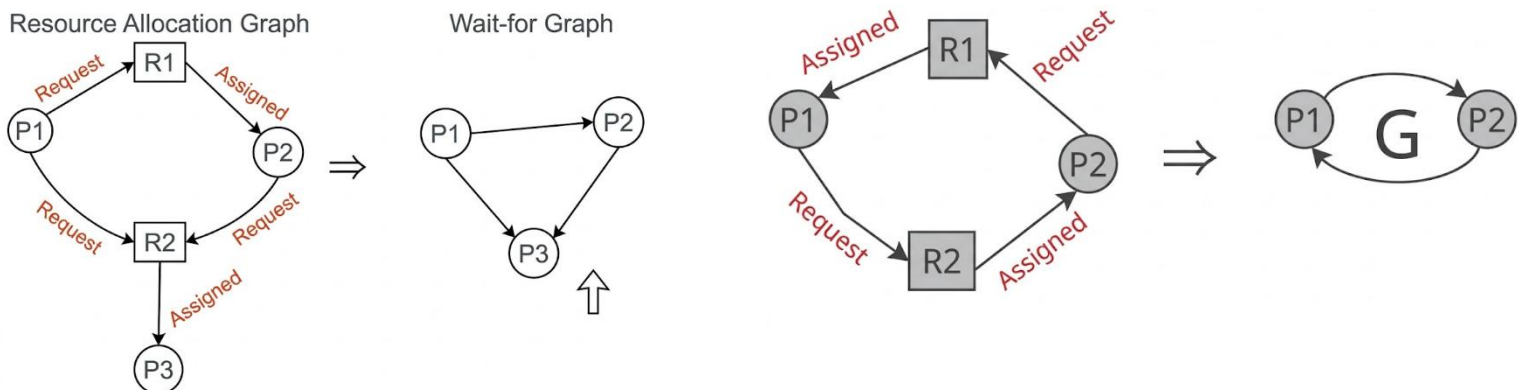
- Mark process as finished
- Add its allocation back:

$$\text{Available} = \text{Available} + \text{Allocation}$$

4. Repeat until:

- All processes finish $\rightarrow$ No deadlock
- Some cannot finish $\rightarrow$ Deadlock exists

1. Wait-for Graph



2. Banker's Algorithm

Total Resources: A = 7, B = 2, C = 6

Process	Allocation (A B C)	Request (A B C)	Available (A B C)
---------	--------------------	-----------------	-------------------

P0	(0, 1, 0)	(0, 0, 0)	(0, 0, 0)
P1	(2, 0, 0)	(2, 0, 2)	(0, 1, 0)
P2	(3, 0, 3)	(0, 0, 0)	(3, 1, 3)
P3	(2, 1, 1)	(1, 0, 0)	(5, 2, 4)
P4	(0, 0, 2)	(0, 0, 2)	(5, 2, 6)
Final Available			(7, 2, 6)

Safe Sequence:

P0 → P2 → P3 → P4 → P1

Total Resources: A = 7, B = 2, C = 6

Process	Allocation (A B C)	Request (A B C)	Available (A B C)
P0	(0, 1, 0)	(0, 0, 0)	(0, 0, 0)
P1	(2, 0, 0)	(2, 0, 2)	(0, 1, 0)
P2	(3, 0, 3)	(0, 0, 1)	—
P3	(2, 1, 1)	(1, 0, 0)	—
P4	(0, 0, 2)	(0, 0, 2)	—